Building a chess opening repertoire under a memory budget

Claude Opus 5.5 (Anthropic)

October 2026

Abstract. A chess player who prepares an opening must decide which moves to memorize: a repertoire is a set of prepared answers, and memory is limited. We formulate this choice as an optimization problem. Games are modelled as a random walk on the game tree whose transition probabilities are the move frequencies observed in a large database of games; the player's prepared moves replace these frequencies at the positions they cover. Under explicit hypotheses on the behaviour of the opponents and of the player outside of preparation, the expected score of a repertoire is a sum over the opponent moves that leave it, weighted by the probability of reaching them. This representation makes the gain of an extension of the repertoire additive over these moves, which reduces the problem to a separable resource allocation problem. Solving it exactly requires data on a number of positions that grows exponentially with the budget; we describe instead a greedy algorithm, with a pruning rule that keeps it exact at each step while querying few positions, and a look-ahead variant. We show by an example that the greedy algorithm can be arbitrarily far from optimal, and illustrate its behaviour on real data.

1 Introduction

Opening preparation is a large part of a chess player's work. A player who has the black pieces and expects the moves 1.e4 e5 must be ready for 2.Nf3, but also, less often, for 2.f4, 2.Nc3, 2.d4 or 2.Bc4; after 2.Nf3 and a reply of their own, again for several moves of the opponent, and so on. Since the number of positions grows exponentially with the depth, no one can prepare everything, and the question is which positions deserve a prepared move.

Chess and mathematics. Chess has attracted mathematicians for a long time, first through combinatorial puzzles on the board. Euler studied the knight's tour, a path of the knight visiting every square exactly once [1], and the problem of placing eight queens so that no two attack each other, posed by Max Bezzel in 1848, was considered by Gauss. Several strong players were mathematicians: Emanuel Lasker, world champion from 1894 to 1921, held a doctorate in mathematics, and his work on ideals in polynomial rings [2] is the origin of the Lasker–Noether theorem; Max Euwe, world champion from 1935 to 1937, was a teacher of mathematics, and used the Thue–Morse sequence to show that the rules of his time allowed infinite games [3].

The game itself became an object of mathematics with Zermelo's lecture at the International Congress of Mathematicians of 1912 [4], often counted as the first theorem of game theory. The statement now attached to his name, refined by Kőnig and Kalmár (see [5] for this history), is that in a finite game of perfect information between two players, such as chess, either one of the players can force a win or both can force at least a draw. The proof is a backward induction on the game tree, the same reasoning that underlies von Neumann's minimax theory [6] and, later, Bellman's dynamic programming, which Bellman himself proposed to apply to chess [7]; retrograde analysis of this kind now produces complete tables of optimal play for endgames with few pieces. The tree is however far too large to be solved: Shannon, in the article that founded computer chess [8], estimated the number of positions at about 10^{43}, and proposed instead to search the tree to a limited depth and to evaluate the positions reached with a heuristic function. This program led to the victory of Deep Blue over Kasparov in 1997 and, combined with learning from self-play, to programs such as AlphaZero [9].

Statistics entered the game through the rating of players: the system of Elo [10], adopted by the international federation in 1970, models the result of a game as a function of the difference of the players' strengths, and Glickman's refinement [11] is used by Lichess, whose games provide the data of this note.

The question studied here differs from Zermelo's and Shannon's in one respect. Their opponent is perfect: the value of a position is a *minimum* over the opponent's replies. A human player faces imperfect opponents whose choices, in a given population, are well described statistically, and the relevant value is an *expectation* over the replies, weighted by how often they are played. The tree is the same, and backward induction remains the main tool.

The question. Two observations make the choice of a repertoire amenable to mathematics. First, large databases of online games give, for every position that occurs often enough, the frequency of each move in a population of players and the results of the games in which it was played. Second, the benefit of a prepared move is a product of two such quantities: the probability that the game reaches the position where it is played, and the increase of the expected score that it brings there, compared with the average result of the population in that position. For instance, a move that raises the expected score by 20 percentage points, against a reply played in 0.1% of the games, raises the player's overall expected score by $0.001 \times 20 = 0.02$ points, whereas a move that raises it by only 3 points, against a reply played in half of the games, raises it by $0.5 \times 3 = 1.5$ points. This is made precise in Lemma 14.

The aim of this note is to describe, with precise definitions and hypotheses, the model and the algorithm used by the program `tools/optimal_repertoire.py` of the open-source application Lirep (lirep.org). Lirep, a web application to build and practise opening repertoires, was written to test whether this quantitative point of view is of practical use to players: it computes the expected score of a repertoire from the database, as in Section 2, and the program studied here suggests which moves to add to it. The algorithm is elementary; the point is rather to make explicit what is being optimized, under which assumptions the objective has a meaning, and where the algorithm is or is not optimal. The note is organized as follows. Section 2 introduces game trees, repertoires and the probabilistic model, and proves a formula for the expected score of a repertoire. Section 3 discusses its estimation from data. Section 4 states the optimization problem and proves a decomposition result. Section 5 presents the greedy algorithm, its pruning rule, an example where it fails, and a look-ahead variant. Section 6 illustrates the method on real data, and Section 7 discusses the hypotheses.

2 Game trees, repertoires and the expected score

2.1 The game tree

We fix a starting position x_0, for example the position after 1.e4 e5, and one of the two sides, called *the player* and denoted by σ; the other side is *the opponent*, denoted by ω.

Definition 1 (Game tree)**.** A *node* is a finite sequence of moves played from x_0, each of which is legal in the position reached by the previous ones; the empty sequence is the *root*, also denoted by x_0. If x is a node and a a legal move after x, we write xa for the node obtained by appending a, and call (x, a) an *edge* from x to its *child* xa. A node is a *leaf* if it has no child. The set T of nodes, ordered by the prefix relation, is a rooted tree, the *game tree*. We denote by N_σ (respectively N_ω) the set of nodes, other than leaves, at which the player (respectively the opponent) is to move. For a node z, we denote by T_z the subtree of nodes having z as a prefix.

A node is a *history*, not a position: two move orders reaching the same position are distinct nodes. The game tree is finite, since the rules of chess (fivefold repetition and the seventy-five-move rule) end

every game after finitely many moves. We will return in Section 3 to the fact that databases record positions rather than histories.

2.2 Repertoires

The player memorizes at most one move at each node where it is their turn, and may prepare answers to several, or all, of the opponent's replies at each node where it is the opponent's turn: this is what makes a repertoire a *strategy* rather than a collection of lines.

Definition 2 (Repertoire)**.** A *repertoire* is a map R, defined on a finite set $D \subset N_\sigma$ and assigning to each $x \in D$ a legal move $R(x)$ at x, such that every node of D is reached by following R. Precisely, let

$$Y(R) = \{x_0 : x_0 \in N_\omega\} \cup \{xR(x) : x \in D\} \tag{1}$$

be the set of opponent nodes reached by the repertoire (the root when the opponent moves first, and the node after each prepared move). We require that every $x \in D$ be either the root, or a child ym of a node $y \in Y(R)$; a node $y \in Y(R)$ may have any number of children in D, one for each prepared reply. The *size* of R is $|R| = \text{card } D$, the number of memorized moves.

By induction on the length of x, the condition means that for every $x \in D$, the path from x_0 to x follows R at each of its player nodes, all of which belong to D.

A repertoire is therefore a subtree of T in which every player node has exactly one child, while opponent nodes may have any number of children. An opponent move m at $y \in Y(R)$ is *answered* if $ym \in D$ and *unanswered* otherwise. The set of unanswered moves,

$$U(R) = \{(y, m) : y \in Y(R),\ m \text{ legal at } y,\ ym \notin D\}, \tag{2}$$

plays the central role in what follows: it is the set of edges through which the game leaves the prepared part of the tree. Figure 1 shows a repertoire for Black after 1.e4 e5 with five memorized moves.

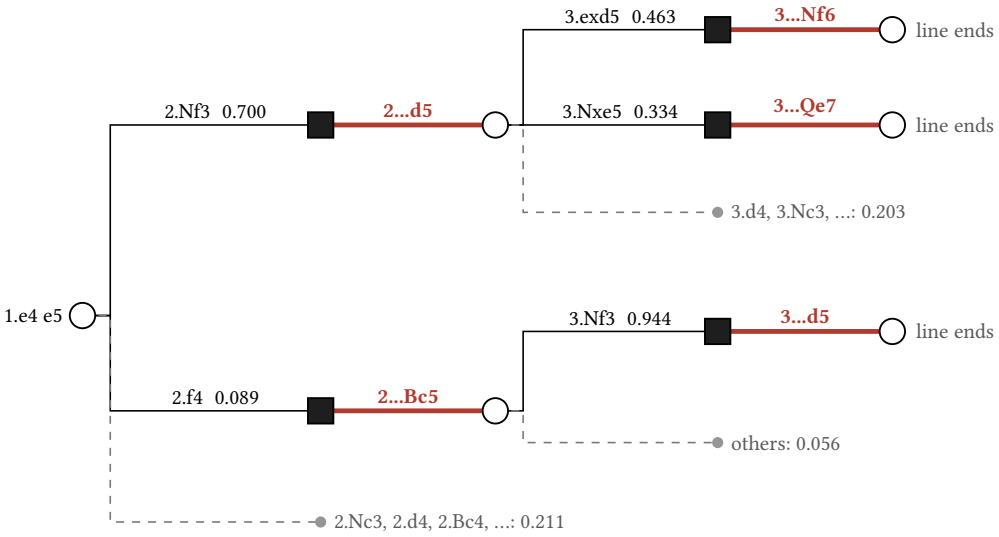


Figure 1: A repertoire R for Black, with root x_0 the node 1.e4 e5 and $|R| = 5$. Circles are opponent nodes (White to move), black squares are player nodes (Black to move). Prepared moves are in red; each label of an opponent move gives its frequency $q(y, m)$ in the database used in Section 6 (blitz and rapid games on Lichess in which the average rating of the two players is at least 2000). Dashed edges group unanswered moves, which belong to $U(R)$; the unanswered replies after 3...Nf6, 3...Qe7 and 3...d5, also in $U(R)$, are not drawn. The set D consists of the five squares; $Y(R)$ consists of the six circles.

2.3 The probabilistic model

The model describes the games of the player against opponents drawn from a fixed population, for example the online games of players with a given rating in a given time control.

Definition 3 (Population law)**.** The *population law* is a probability measure $\mathbf{P}$ on the set of leaves of T (the complete games starting from x_0), together with, for each complete game, its result $Z \in \{0, \frac{1}{2}, 1\}$ from the player's point of view (loss, draw, win). For a node x, we write $\mathbf{P}(x)$ for the probability that the game passes through x, and, when $\mathbf{P}(x) > 0$,

$$q(x,a) = \frac{\mathbf{P}(xa)}{\mathbf{P}(x)}, \quad s(x,a) = \mathbf{E}[Z \mid \text{the game passes through } xa], \tag{3}$$

and $s(x) = \mathbf{E}[Z \mid \text{the game passes through } x]$. Thus $q(x,\cdot)$ is the distribution of the move played at x in the population, and $s(x,a)$ is the *score* of the move a at x: the mean result, for the player, of the games in which it was played.

The result Z is a function of the complete game rather than of the final position, since games also end by resignation, agreement or on time; the reader may think of the end of the game, with its result, as a last move drawn from the population law. Since the nodes are histories, the conditional law of the continuation of a game, given that it passes through x, is the law of the random walk started at x with transition probabilities q; we use this Markov property repeatedly.

We now describe the games of a player who has memorized a repertoire R. The model rests on two hypotheses.

Hypothesis 4 (Opponents)**.** At every opponent node x, the opponent plays the move a with probability $q(x,a)$. In particular, the opponents' choices do not depend on the player's repertoire.

Hypothesis 5 (Play outside of preparation)**.** At every player node $x \notin D$, the player plays the move a with probability $q(x,a)$, and the result of a game, given its moves, has the same law as in the population.

Hypothesis 5 says that outside of their preparation, the player performs like the average member of the population who reached the same node. Under Hypothesis 4 and Hypothesis 5, the games of the player are distributed according to the law $\mathbf{P}_R$ of the random walk on T started at x_0 with transitions

$$k_R(x,a) = \begin{cases} 1 & \text{if } x \in D \text{ and } a = R(x), \\ 0 & \text{if } x \in D \text{ and } a \neq R(x), \\ q(x,a) & \text{if } x \notin D \end{cases} \tag{4}$$

and with the population's law of the result given the moves. We write $\mathbf{E}_R$ for the expectation under $\mathbf{P}_R$ and $\pi_R(x) = \mathbf{P}_R\begin{pmatrix}\text{the game} \\ \text{passes through}\end{pmatrix} x$. For an edge $e = (y,m)$, we write $\pi_R(e) = \pi_R(ym)$ and $s(e) = s(y,m)$. To ensure that all the quantities above are defined, we assume throughout that $\mathbf{P}(x) > 0$ for every $x \in D \cup Y(R)$, and that no node of $D \cup Y(R)$ is a leaf; in practice, prepared lines are short and only moves that occur in the database are prepared.

Definition 6 (Value of a repertoire)**.** The *value* of a repertoire R is $V(R) = \mathbf{E}_R[Z]$, the expected score of the player under Hypothesis 4 and Hypothesis 5.

The following representation is the basis of everything that follows.

Proposition 7 (Expected score as a sum over unanswered moves)**.** *Let R be a repertoire with $D \neq \varnothing$ or $x_0 \in N_\omega$. Then the edges of $U(R)$ are crossed by disjoint sets of games, one of them is crossed $\mathbf{P}_R$-almost surely, and*

$$V(R) = \sum_{e \in U(R)} \pi_R(e)\, s(e), \qquad \sum_{e \in U(R)} \pi_R(e) = 1. \tag{5}$$

Moreover, for $e = (y, m) \in U(R)$, the probability of crossing e is $\pi_R(e) = \pi_R(y)q(y, m)$, where $\pi_R(y)$ is the product of the frequencies q of the opponent moves on the path from x_0 to y.

Proof. Under $\mathbf{P}_R$, the game follows R at the nodes of D and the population frequencies at the opponent nodes. By the definition of a repertoire, as long as the game stays in $D \cup Y(R)$, it alternates between a node $y \in Y(R)$ and, if the opponent's move m is answered, a node $ym \in D$ followed by $ymR(ym) \in Y(R)$. Since D is finite and no node of $D \cup Y(R)$ is a leaf, the game almost surely reaches a node $y \in Y(R)$ at which the opponent plays an unanswered move m, and it does so only once, since the descendants of ym are not in D (every node of D has all its player-ancestors in D). This proves that the events {the game crosses e}, $e \in U(R)$, form a partition up to a null set, hence the second identity. The product formula for π_R follows from the definition of k_R, since the player's moves on the path have probability one.

Fix $e = (y, m) \in U(R)$ with $\pi_R(e) > 0$. No descendant of ym is in D, so after crossing e the walk under $\mathbf{P}_R$ uses the transitions q only. By the Markov property, the conditional law under $\mathbf{P}_R$ of the continuation given that the game crosses e is equal to the conditional law under $\mathbf{P}$ given that the game passes through ym; hence $\mathbf{E}_R[Z \mid e \text{ crossed}] = s(e)$. The first identity follows from the law of total expectation. □

For the repertoire of Figure 1, (5) is a sum over the unanswered moves after 1.e4 e5, after 2.Nf3 d5, after 2.f4 Bc5, and after each of the three last prepared moves. Table 1 gives the numerical values.

Remark 8 (Lines ending with a prepared move)**.** Let $x \in D$ and $y = xR(x)$ be such that no opponent move at y is answered: the line *ends* with the prepared move $R(x)$. Then, by the law of total expectation,

$$\sum_m q(y, m)s(y, m) = s(x, R(x)). \tag{6}$$

In (5), the terms of all the moves m at such a node y can therefore be replaced by the single term $\pi_R(x)s(x, R(x))$: the value of a line that ends with a prepared move is the score of that move. This is the form used by Lirep, and we use it in Table 1. The two forms are equal in the model; they differ when s and q are replaced by estimates, since (6) need not hold for the estimates (Section 3). The form with $s(x, R(x))$ uses the sample of the games in which the prepared move itself was played, which is the more natural one.

Remark 9 (Backward induction)**.** (5) can be evaluated by recursion from the leaves of the repertoire. For $y \in Y(R)$, define $v_R(y)$ as the conditional expectation of Z under $\mathbf{P}_R$ given that the game passes through y. Then

$$v_R(y) = \sum_m q(y, m) \cdot \begin{cases} v_R(ymR(ym)) & \text{if } ym \in D, \\ s(y, m) & \text{otherwise} \end{cases} \tag{7}$$

and $V(R) = v_R(x_0)$ if $x_0 \in N_\omega$, and $V(R) = v_R(x_0R(x_0))$ otherwise. Evaluating $V(R)$ requires the frequencies and scores of the opponent moves at each node of $Y(R)$, that is, one query to the database per prepared move.

3 Estimation from a database of games

The quantities q and s are not known; they are estimated from a database. The program uses the opening explorer of Lichess, which returns, for a position and a population of games (a range of ratings and time controls), the list of the moves played in it, with, for each move a, the numbers n_W, n_D and n_B of games won by White, drawn, and won by Black. We write $n(x,a) = n_W + n_D + n_B$ and $p(x,a)$ for the number of points obtained by the player in these games ($n_W + n_D/2$ for White).

Definition 10 (Estimators)**.** Let $\kappa \geq 0$. The estimated frequency and score of the move a at x are

$$\hat{q}(x,a) = \frac{n(x,a)}{\sum_b n(x,b)}, \qquad \hat{s}(x,a) = \frac{p(x,a) + \kappa/2}{n(x,a) + \kappa}, \tag{8}$$

where the sum runs over the moves listed by the database at x.

The program uses $\kappa = 20$. The parameter κ pulls the estimated score toward $1/2$, as if κ additional drawn games had been observed; its effect is negligible when $n(x,a)$ is large and substantial when it is of the order of κ. It has a standard Bayesian interpretation.

Lemma 11. *Let the results of the n games in which a move was played be independent with law $(\theta_W, \theta_D, \theta_L)$ on $\{1, 1/2, 0\}$, and put on θ a Dirichlet prior with parameters (α, β, α), with $2\alpha + \beta = \kappa$. Then the posterior mean of the expected score $\theta_W + \theta_D/2$ is $(p + \kappa/2)/(n + \kappa)$, where p is the number of points scored.*

Proof. The Dirichlet prior is conjugate to the multinomial likelihood: the posterior is Dirichlet with parameters $\big(\alpha + n_1, \beta + n_{1/2}, \alpha + n_0\big)$, where n_r is the number of games with result r, and its mean is the vector of these parameters divided by $n + \kappa$. Hence the posterior mean of $\theta_W + \theta_D/2$ is $\big(n_1 + \alpha + \big(n_{1/2} + \beta\big)/2\big)/(n + \kappa) = (p + \kappa/2)/(n + \kappa)$, since $\alpha + \beta/2 = \kappa/2$. □

Three sources of error should be kept in mind.

Sampling error. Since $Z \in [0,1]$, its variance is at most $1/4$, and the standard deviation of the empirical score of a move played in n independent games is at most $1/(2\sqrt{n})$: about 0.05 for $n = 100$ and 0.016 for $n = 1000$.

Selection bias. The algorithm chooses, among several candidate moves, the one with the largest estimated score. If $\hat{s}_1, \ldots, \hat{s}_k$ are unbiased estimators of $s_1, \ldots, s_k$, then $\mathbf{E}[\max_i \hat{s}_i] \geq \max_i s_i$, with a gap that grows with the variance of the estimators and with k: the maximum of noisy estimates is optimistic. This is why the program only considers, at each player node, the $k = 6$ most played moves among those played in at least $n_{\min} = 100$ games, and uses $\kappa > 0$.

Positions versus histories. The database is indexed by positions, so the data at a node x aggregates all the games reaching the position of x, by any move order. The model of Section 2 is then applied with $q(x,\cdot)$ and $s(x,\cdot)$ depending on x through its position only, which is a modelling assumption. Moreover, the database lists a limited number of moves at each position (about a dozen); the frequencies are normalized over the listed moves, and the omitted moves, which are rare, are ignored. For these reasons (6) holds only approximately for the estimates.

In the next two sections, the results are stated for the model of Section 2, with the exact quantities q and s. The program applies the same formulas with the estimates $\hat{q}$ and $\hat{s}$ in their place, with the form of Remark 8 for the lines ending with a prepared move; the identities that rely on (6) then hold only approximately.

4 The optimization problem

The player has a budget of B moves to memorize. Two constraints restrict the repertoires considered, both motivated by the estimation errors of Section 3. For each player node x, a finite set $A(x)$ of *candidate moves* is given (in the program, the most played moves with at least $n_{\min}$ games), and a threshold $\rho \in [0, 1]$ is given (in the program, $\rho = 0.002$).

Definition 12 (Admissible repertoires and the problem)**.** A repertoire R is *admissible* if $R(x) \in A(x)$ and $\pi_R(x) \geq \rho$ for every $x \in D$: only candidate moves are prepared, and only at nodes reached in a proportion at least ρ of the games. Given an admissible repertoire R_0 (possibly empty) and a budget $B \in \mathbb{N}$, the problem is

$$\text{maximize } V(R) \quad \text{over admissible repertoires } R \text{ extending } R_0 \text{ with } |R| \leq |R_0| + B. \tag{9}$$

Here R *extends* R_0 if $D_0 \subset D$ and $R = R_0$ on D_0.

Starting from a non-empty R_0 corresponds to completing an existing repertoire, which is the common situation in practice. Since T and the sets $A(x)$ are finite, (9) has a solution. The next result shows that it has a separable structure.

For a node $z \in N_\sigma$ and a repertoire S whose domain is contained in T_z and which is a repertoire for the game started at z (in the sense of Definition 2 with x_0 replaced by z), we denote by

$$V_z(S) = \mathbf{E}[Z \mid \text{the game passes through } z, \text{the player follows } S] \tag{10}$$

its value in the game started at z, with the convention $V_z(\varnothing) = s(e)$ when z is the child of the edge e.

Theorem 13 (Decomposition)**.** *Let R be a repertoire with $D \neq \varnothing$ or $x_0 \in N_\omega$, and R' a repertoire extending R, with domain D'. For each $e = (y, m) \in U(R)$, let $z_e = ym$ and let S_e be the restriction of R' to the nodes of T_{z_e}. Then*

$$V(R') - V(R) = \sum_{e \in U(R)} \pi_R(e)\big(V_{z_e}(S_e) - s(e)\big), \qquad |R'| - |R| = \sum_{e \in U(R)} |S_e|. \tag{11}$$

Proof. We first check that every node $x \in D' \setminus D$ lies in exactly one of the subtrees T_{z_e}. They are disjoint: if z_e were a proper prefix of $z_{e'}$, the parent y' of $z_{e'}$, which belongs to $Y(R)$, would be a descendant of z_e, hence of the form $x' R(x')$ with $x' \in D \cap T_{z_e}$; since the player nodes on the path from x_0 to x' belong to D, so would z_e, a contradiction. Now let $x \in D' \setminus D$, and let u be the last node of the path from x_0 to x that belongs to $D \cup Y(R)$ (the root belongs to this set by the assumption on R). If $u \in D$, the next node of the path is $uR'(u) = uR(u) \in Y(R)$, a contradiction; so $u \in Y(R)$, the next node um is not in D, and $(u, m) \in U(R)$ with $x \in T_{um}$. This proves the identity on the sizes.

The transitions k_R and $k_{R'}$ coincide outside of $D' \setminus D$, and $D' \setminus D$ is contained in the union of the T_{z_e}. Hence the two walks coincide until the edge of $U(R)$ that the game crosses, and $\mathbf{P}_{R'}(\text{the game crosses } e) = \pi_R(e)$. Given that e is crossed, the continuation under $\mathbf{P}_{R'}$ follows S_e at the nodes of its domain and the population frequencies elsewhere, so the conditional expectation of Z is $V_{z_e}(S_e)$. By the law of total expectation, $V(R') = \sum_{e \in U(R)} \pi_R(e) V_{z_e}(S_e)$, and subtracting (5) gives the result. □

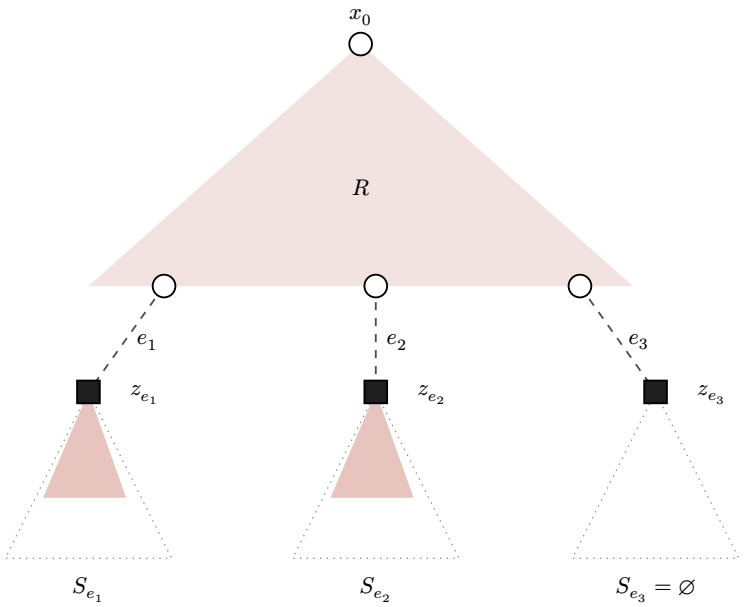


Figure 2: Theorem 13. An extension R' of R adds, below each unanswered move $e \in U(R)$, a repertoire S_e of the subgame starting at the child z_e of e (dotted triangle: the subtree T_{z_e}). The gain is the sum of the gains $\pi_R(e)\big(V_{z_e}(S_e) - s(e)\big)$, and the number of added moves is the sum of the $|S_e|$.

Theorem 13, illustrated in Figure 2, reduces (9) to a separable resource allocation problem. For $e \in U(R_0)$ and $c \in \mathbb{N}$, let

$$F_e(c) = \max\Big\{V_{z_e}(S) - s(e) : S \text{ admissible below } z_e,\ |S| \le c\Big\}, \tag{12}$$

where "admissible below z_e" means that R_0 extended by S is admissible. Then the optimal value of (9) is

$$V(R_0) + \max\left\{\sum_{e \in U(R_0)} \pi_{R_0}(e) F_e(c_e) : c_e \in \mathbb{N},\ \sum_e c_e \le B\right\}. \tag{13}$$

Both (12) and (13) can be solved exactly by dynamic programming on the tree, in the manner of a knapsack problem (see Section 5.3 for the recursion). The difficulty is not the computation but the data: evaluating $F_e(c)$ requires the frequencies and scores at every node that a repertoire of size c below z_e can reach. With k candidate moves per player node and b opponent moves per opponent node that pass the threshold ρ, there are of the order of $(kb)^c$ such nodes. Each node costs one query to the database, and the public explorer of Lichess answers about one query per second. For $B = 50$, the exact solution is out of reach, and an algorithm that queries few nodes is needed.

5 The greedy algorithm

5.1 Description

The greedy algorithm adds one move at a time, each time the one whose addition increases V the most. By Theorem 13, the effect of a single move is explicit.

Lemma 14 (Gain of one move)**.** *Let R be a repertoire, $e = (y, m) \in U(R)$, $z = ym$, and $a \in A(z)$. Let $R + (e, a)$ be the repertoire extending R by $R(z) = a$. Then*

$$V(R + (e, a)) - V(R) = \pi_R(e)(s(z, a) - s(e)). \tag{14}$$

Moreover, the identity $\sum_a q(z,a)s(z,a) = s(e)$ holds: the gain is the probability of the move e times the excess of the score of a over the average score of the population at z. In particular, if the move of largest score at z is a candidate, the largest gain at e is nonnegative.

Proof. Apply Theorem 13 with $S_e = \{z \mapsto a\}$ and $S_{e'} = \varnothing$ for $e' \neq e$. After the move a, no move is prepared, so $V_z(S_e) = \mathbf{E}[Z \mid \text{the game passes through } za] = s(z,a)$. The identity $\sum_a q(z,a)s(z,a) = s(e)$ is the law of total expectation; since a maximum is not smaller than an average, the last claim follows. □

The data needed to compute $\pi_R(e)$ and $s(e)$ are those at the node y, which belongs to $Y(R)$ and has been queried already when evaluating $V(R)$; only the score $s(z,a)$ requires a query at the new node z. The bound $s(z,a) \leq 1$ gives the following upper bound, which does not require this query.

Lemma 15 (Bound)**.** *For every $e \in U(R)$ and $a \in A(z_e)$, $V(R + (e,a)) - V(R) \leq \beta(e) := \pi_R(e)(1 - s(e))$.*

The algorithm sorts the unanswered moves by decreasing bound and examines them in this order, querying the database at z_e; it stops as soon as the bound of the next move does not exceed the best gain found so far. Since the bound is an upper bound, the move found is a maximizer of the gain over all pairs (e,a), although only a few nodes are queried. This is a branch-and-bound search of depth one.

Input: an admissible repertoire R_0, a budget B, candidate sets $A(\cdot)$, a threshold ρ.
Output: an admissible repertoire R extending R_0 with $|R| \leq |R_0| + B$.

1. $R \leftarrow R_0$.
2. **while** $|R| < |R_0| + B$ **do**
   1. Compute $\pi_R(e)$ and $s(e)$ for every $e \in U(R)$ such that $\pi_R(e) \geq \rho$, and sort these moves by decreasing $\beta(e) = \pi_R(e)(1 - s(e))$.
   2. $g^* \leftarrow -\infty$.
   3. **for** e in this order **do**
      1. **if** $\beta(e) \leq g^*$ **then break**.
      2. Query the database at z_e. For each $a \in A(z_e)$, compute $g = \pi_R(e)(s(z_e,a) - s(e))$; if $g > g^*$, set $g^* \leftarrow g$ and $(e^*, a^*) \leftarrow (e,a)$.
   4. **if** $g^* \leq 0$ **then return** R.
   5. $R \leftarrow R + (e^*, a^*)$.
3. **return** R.

Algorithm 1: The greedy algorithm with pruning.

When the root is a player node and $D_0 = \varnothing$, the first move is chosen among the candidates at the root by its score, since then $V(\{x_0 \mapsto a\}) = s(x_0,a)$. The program also accepts a target value instead of a budget, in which case the loop stops when $V(R)$ reaches the target; since moves are added one at a time, the value after each addition is available, and the output is a whole sequence of nested repertoires, one per budget.

5.2 The greedy algorithm is not optimal

The greedy algorithm only sees one move ahead. A move whose own gain is small, but which makes a large gain possible at the next move, is missed.

Example 16. Consider the tree of Figure 3. The root x_0 is an opponent node with two moves m_1, m_2, each of frequency $1/2$ and of score $1/2$. At $z_1 = x_0 m_1$ the only candidate is a, with $s(z_1,a) = 1/2$; after it, the opponent plays r_1 or r_2, each with frequency $1/2$, with scores 0.3 and 0.7. At $z_1 a r_1$ the only candidate is b, with score 1. At $z_2 = x_0 m_2$, the only candidate is c,

with $s(z_2, c) = 1/2 + \varepsilon$, $0 < \varepsilon < 0.35$. There are no other candidates, and $\rho = 0$. These numbers satisfy (6) (at $z_1 a$, $(0.3 + 0.7)/2 = 1/2$).

With $B = 2$, the greedy algorithm compares the gains $1/2 \cdot (1/2 - 1/2) = 0$ for a and $\varepsilon/2$ for c, and chooses c. At the second step, the only addition is a, with gain 0, and the algorithm stops: its gain is $\varepsilon/2$. The repertoire $\{z_1 \mapsto a, z_1 a r_1 \mapsto b\}$ has gain $1/2 \cdot 1/2 \cdot (1 - 0.3) = 0.175$. As $\varepsilon \to 0$, the ratio of the greedy gain to the optimal gain tends to zero.

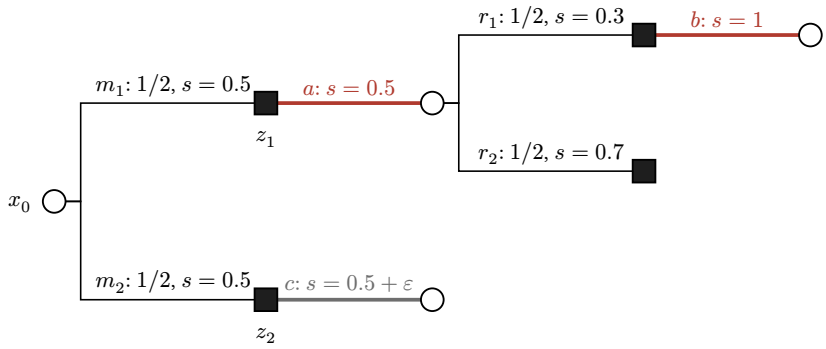


Figure 3: Example 16. Edge labels give the frequency of opponent moves and the score of every move. With two moves to memorize, the optimal repertoire is $\{a, b\}$ (red), of gain 0.175; the greedy algorithm first chooses c (grey), of gain $\varepsilon/2$, and then cannot use b.

In the language of (13), the greedy algorithm fails because the functions F_e need not be concave: in Example 16, the function F_e with $e = (x_0, m_1)$ satisfies $F_e(0) = F_e(1) = 0$ and $F_e(2) = 0.35$. For separable problems with concave nondecreasing functions, the greedy allocation of one unit at a time is optimal (a classical result on resource allocation going back to Gross, see [12] and the monograph [13]); here the functions F_e are only known to be nondecreasing, and the set $U(R)$ changes as moves are added.

5.3 Look-ahead

The program offers a variant with a parameter $K \geq 1$: for each unanswered move e examined, it computes the best repertoires below z_e with at most c moves, that is, $F_e(c)$ in (12), for $c = 1, \ldots, K$, and among all pairs (e, c) it adds the repertoire with the largest *gain per move* $\pi_R(e) F_e(c)/c$, all of its moves at once. For $K = 1$ this is Algorithm 1. In Example 16, $K = 2$ finds the gain per move $0.175/2 > \varepsilon/2$ and returns the optimal repertoire.

The values $F_e(c)$, $c \leq K$, are computed by the following recursion, from the leaves up. For a player node z reached by the opponent move e, and an opponent node y, define, for $c = 0, \ldots, K$,

- $G_z(c)$: the largest value of a repertoire of size at most c below z, conditionally on reaching z; thus $G_z(0) = s(e)$;
- $H_y(c)$: the same quantity at the opponent node y, for repertoires of size at most c prepared after the moves at y.

Then $H_y(0) = \sum_m q(y, m) s(y, m)$, and, for $c \geq 1$,

$$G_z(c) = \max\left(G_z(0),\ \max_{a \in A(z)} H_{za}(c-1) \right), \qquad H_y(c) = \max_{c_m \geq 0,\ \sum_m c_m \leq c} \sum_m q(y, m) G_{ym}(c_m), \tag{15}$$

where the inner maximum is a knapsack over the opponent moves at y, solved by adding the moves one at a time in $O(K^2)$ operations each, and where we set $G_{ym}(c) = G_{ym}(0)$ for all c if the probability of reaching ym is below ρ. Then $F_e(c) = G_{z_e}(c) - s(e)$ for $c \leq K$. In the model, the identity $H_{za}(0) = s(z, a)$ holds by (6); since the estimates satisfy it only approximately, the program

uses $s(z, a)$, the score of the move a itself, as the value of a line ending with a, in accordance with Remark 8. The bound of Lemma 15 still applies, since $\pi_R(e)F_e(c)/c \le \pi_R(e)F_e(c) \le \beta(e)$, and the pruning rule is unchanged.

The cost of the look-ahead is that of the data: examining e requires the nodes reachable by K prepared moves below z_e, of the order of $(kb)^K$ queries. The choice by gain per move is a heuristic, as for the knapsack problem, and the look-ahead variant is not optimal either: an example of the same type as Example 16, with a chain of $K + 1$ moves, defeats it.

6 A numerical illustration

We consider Black after 1.e4 e5, with $R_0 = \varnothing$, and the population of blitz and rapid games on Lichess in which the average rating of the two players is at least 2000 (about 8×10^7 games through 1.e4 e5). The parameters are those described above: $\kappa = 20$, at most $k = 6$ candidate moves per node, each played in at least $n_{\min} = 100$ games, and $\rho = 0.002$. The value of the empty repertoire is the score of Black in the population after 1.e4 e5, $V(\varnothing) = 47.63\%$.

Where the game leaves R	π_R	mean score	contribution
2.Nc3, 2.d4, 2.Bc4, ... (10 moves)	0.2113	0.4691	0.0991
2.Nf3 d5 3.d4, 3.Nc3, ... (10 moves)	0.1423	0.5102	0.0726
2.f4 Bc5, moves other than 3.Nf3 (11 moves)	0.0050	0.5313	0.0026
line ends: 2.Nf3 d5 3.exd5 Nf6	0.3240	0.5300	0.1717
line ends: 2.Nf3 d5 3.Nxe5 Qe7	0.2339	0.5353	0.1252
line ends: 2.f4 Bc5 3.Nf3 d5	0.0835	0.5571	0.0465
Total	1.0000		0.5178

Table 1: The terms of (5) for the repertoire of Figure 1, grouped, in the form of Remark 8: a group of unanswered moves contributes the sum of $\pi_R(e)s(e)$ over its moves, and a line ending with a prepared move $R(x)$ contributes $\pi_R(x)s(x, R(x))$. The middle column is the contribution divided by the probability. The value is $V(R) = 51.78\%$.

The first five moves chosen by Algorithm 1 are those of Figure 1, in the order 2.Nf3 d5, 3.Nxe5 Qe7, 3.exd5 Nf6, 2.f4 Bc5, 3.Nf3 d5. The first one illustrates Lemma 14: the move 2.Nf3 is played in 70.0% of the games, Black's average score after it is $s(e) = 47.6\%$, and 2...d5 scores $s(z, a) = 51.4\%$, so that the gain is $0.700 \times (0.514 - 0.476) \approx 2.65$ points. Table 1 shows how the value 51.78% of the resulting repertoire splits over the ways of leaving it.

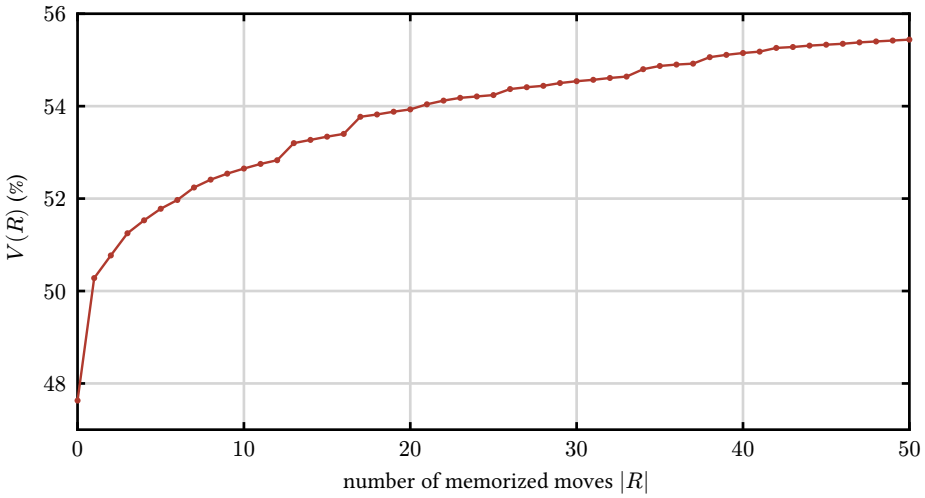


Figure 4: Value of the repertoires produced by Algorithm 1 for Black after 1.e4 e5, as a function of the number of moves (one run of 50 steps). The gains decrease overall but not monotonically: the 13th move (3…d6 after 2.d4 Nc6 3.dxe5) gains 0.37 points, while the 12th (2…Nc6 against 2.d4), which made it possible, gained 0.08.

Figure 4 shows the value along the run: after 50 moves, $V(R) = 55.44\%$, an increase of 7.8 points. The run queried the database at 193 nodes, a small fraction of the nodes that an exact solution of (9) would require. The sequence of gains decreases overall, which reflects the fact that the first moves answer the most frequent opponent moves. It is not monotone, however, and the exceptions have the structure of Example 16: a move of small gain opens a branch where a larger gain becomes available. In this run, the greedy algorithm took these moves for their own small gain, but nothing guarantees that it does so in general.

The first move of the run, the gambit 2…d5 against 2.Nf3, is generally considered objectively dubious. Its good score in the population is a reminder of what $s(z, a)$ measures, which is the subject of the next section.

7 Discussion of the hypotheses

Hypothesis 4 is natural if the population is chosen to match the player's actual opponents (rating range and time control). It does exclude opponents who adapt to the player's repertoire, for instance by preparing against it.

Hypothesis 5 is the strongest assumption. The score $s(z, a)$ is the mean result of the players of the population who *chose* to play a at z. If those who choose a rare move are, more often than others, players who know it well, then $s(z, a)$ overestimates the score of a player who adopts the move because an algorithm recommended it. In statistical language, $s(z, a)$ is an observational quantity, and the choice of a is not randomized. The assumption is more plausible for the opponent moves, whose frequencies are what the player will actually face, than for the scores of the player's own rare moves; the threshold $n_{\min}$ and the shrinkage parameter κ limit, but do not remove, this effect.

The objective itself can be changed without changing the algorithm. If the score s is replaced, at the edges where the game leaves the repertoire, by another function g, for instance a chess engine's evaluation of the position converted to an expected score by a logistic function, then (5) with g in place of s defines an objective, and Theorem 13, Lemma 14 (without the claim on the sign of the gain) and the algorithms carry over. What is lost is the probabilistic interpretation as an expected score, and (6), so that the value assigned to a line ending with a prepared move becomes a modelling choice. The program offers such objectives; we have restricted the exposition to the expected score, for which the model is the cleanest.

Finally, two simplifications deserve mention. The model assumes that the player always remembers the prepared move; a probability of forgetting could be included by mixing $k_R(x,\cdot)$ with $q(x,\cdot)$ at the nodes of D, which would make the gain of a move proportional to the probability of remembering it. And the budget counts memorized moves uniformly, whereas moves on frequent lines are rehearsed more often and are easier to retain.

References

[1] L. Euler, “Solution d'une question curieuse qui ne para\it soumise à aucune analyse,” *Mémoires de l'Académie Royale des Sciences et Belles-Lettres de Berlin*, vol. 15, 1766.

[2] E. Lasker, “Zur Theorie der Moduln und Ideale,” *Mathematische Annalen*, vol. 60, pp. 20–116, 1905.

[3] M. Euwe, “Mengentheoretische Betrachtungen über das Schachspiel,” *Proceedings of the Koninklijke Akademie van Wetenschappen te Amsterdam*, vol. 32, pp. 633–642, 1929.

[4] E. Zermelo, “Über eine Anwendung der Mengenlehre auf die Theorie des Schachspiels,” in *Proceedings of the Fifth International Congress of Mathematicians (Cambridge, 1912)*, Cambridge University Press, 1913, pp. 501–504.

[5] U. Schwalbe and P. Walker, “Zermelo and the early history of game theory,” *Games and Economic Behavior*, vol. 34, no. 1, pp. 123–137, 2001.

[6] J. von Neumann, “Zur Theorie der Gesellschaftsspiele,” *Mathematische Annalen*, vol. 100, pp. 295–320, 1928.

[7] R. Bellman, “On the application of dynamic programming to the determination of optimal play in chess and checkers,” *Proceedings of the National Academy of Sciences*, vol. 53, no. 2, pp. 244–246, 1965.

[8] C. E. Shannon, “Programming a computer for playing chess,” *Philosophical Magazine*, vol. 41, no. 314, pp. 256–275, 1950.

[9] D. Silver *et al.*, “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play,” *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[10] A. E. Elo, *The Rating of Chessplayers, Past and Present*. New York: Arco, 1978.

[11] M. E. Glickman, “Parameter estimation in large dynamic paired comparison experiments,” *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, vol. 48, no. 3, pp. 377–394, 1999.

[12] B. Fox, “Discrete optimization via marginal analysis,” *Management Science*, vol. 13, no. 3, pp. 210–216, 1966.

[13] T. Ibaraki and N. Katoh, *Resource Allocation Problems: Algorithmic Approaches*. Cambridge, MA: MIT Press, 1988.